

Controller Synthesis for High-Speed Differential Wheeled Robot

1st José Pedro Carvalho

Department of Electrical and Computer Engineering
Faculdade de Engenharia da Universidade do Porto
Porto, Portugal
jpcarvalho@fe.up.pt

2nd António Pedro Aguiar

Department of Electrical and Computer Engineering
Faculdade de Engenharia da Universidade do Porto
Porto, Portugal
pedro.aguiar@fe.up.pt

Abstract—This document is the final report of the Non Linear Control Course at PDEEC.

I. INTRODUCTION

The rest of the document is structured as follows: Section II formulates the path following problem and derives two kinodynamic models for the differential wheeled robot. Section III explores the design and synthesis of Proportional Integral and Derivative (PID) and Model Reference Adaptive Control (MRAC) controllers for the wheel's velocity. Subsequently Section IV formulates four different path-following controllers based on: Linear Quadratic Regulator, Sliding Mode Control, Non Linear Control, Proximal Policy Optimization, giving design choices for each of the approach. This is followed by Section V with the results of both the low-level controller and the high-level controller and finally in Section VI the main conclusions and future work are presented.

II. PROBLEM FORMULATION

In this section, we will formulate the class of path-following problems that will be considered in this paper. First, we will consider a framework where a mobile robot is modeled by a continuous-time dynamical system of the form:

$$\begin{aligned} \dot{x}(t) &= f(x(t), u(t)), & x(0) &= x_0 \\ y(t) &= h(x(t)), \end{aligned} \quad (1)$$

where the vector $x(t) \in \mathbb{R}^n$ denotes the state, x_0 the initial condition, $u(t) \in \mathbb{R}^n$ the input control signal, and $y(t) \in \mathbb{R}^q$ the output signal that will typically correspond to the robot's pose in respect to an inertial coordinate frame.

We are interested in the class of problems where a robot described by the system in equation (1) must follow a geometric path $y_d(\gamma) : \mathbb{R} \rightarrow \mathbb{R}^q$ parameterized by $\gamma \in \mathbb{R}$ and $v_d \in \mathbb{R}$ is the desired speed assignment. As such, the goal of the path-following problem is to design a feedback control law such that all relevant closed-loop signals are bounded and the path-following error $\|y(t) - y_d(\gamma(t))\|$ converges to zero as $t \rightarrow \infty$, and the parameter γ satisfies the speed assignment $\dot{\gamma} \rightarrow v_d$ as $t \rightarrow \infty$.

In this document, we will address only the case where the path is a straight line, and as such, $\gamma(t) = \bar{x}(t)$, $\forall t$, where $\bar{x}$

Identify applicable funding agency here. If none, delete this.

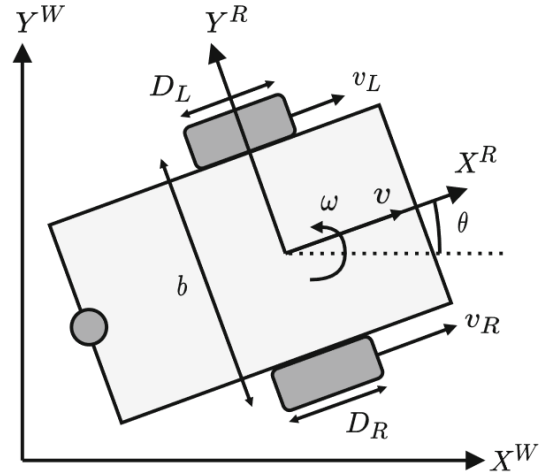


Fig. 1: Example of Differential Wheeled Robot [?]

is the x coordinate of the robot on the global frame, and thus $y_d(\bar{x}(t)) = [\bar{x}, 0]^T$, meaning that the robot will always follow the line corresponding to the x -axis $y = 0$.

A. Robot Kinodynamic Model

The robot used in this work will be a differential wheeled robot with two actuators in the form of motors. A typical example of such a robot can be visualized in Figure 1.

The robot kinematics are nonlinear and have non-holonomic motion constraints, meaning that the 3 degrees of freedom (x, y, θ) are not decoupled from one another. The forward kinematics can be written as:

$$\underbrace{\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}}_J = \underbrace{\begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix}}_J \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (2)$$

where v is the linear velocity on the robot inertial frame, and ω is its angular velocity. These velocities can be related to the

wheel velocities through the inverse kinematics:

$$\begin{bmatrix} v_L \\ v_R \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & -\frac{b}{2} \\ 1 & \frac{b}{2} \end{bmatrix}}_W \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3)$$

in which b is the robots' width, v_L , and v_R are the velocities of the left and right wheels, respectively. The wheels' angular speeds are calculated by the relation with the wheel radius r :

$$\omega_w = \frac{v}{r}, \quad (4)$$

and the subscript w will be used whenever referring to a generic wheel. Considering that the motors operate outside their deadzone, the dynamical model can be approximated as a linear model with two poles. After performing system identification on the system, it was concluded that the pole associated with the electrical current is much smaller than the mechanical pole, and therefore, to simplify the model, it will be ignored. Thus, the motors will be described by a first-order model with the following transfer function on the Laplace domain:

$$G(s) = \frac{W(s)}{U(s)} = \frac{A}{\tau s + 1}, \quad (5)$$

where A is the transfer function gain, τ is the system's open-loop time constant and $U(s)$ is the voltage signal, or the control variable.

The goal of a generic closed-loop controller $H(s)$ will be to make it so that the system tracks the reference signal $R(s)$. In this section, we will simplify the controller synthesis by considering a generic controller that imposes the closed-loop dynamics described by:

$$\frac{1}{\tau' s + 1}, \quad (6)$$

Which is a unitary-gain first-order system with a time constant τ' . Considering that there is no wheel slippage or actuator saturation, robot velocities v, ω also follow the closed-loop dynamics of equation (6), and thus the model can be extended as:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 0 & \sin \theta \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & \frac{-1}{\tau'} & 0 \\ 0 & 0 & 0 & 0 & \frac{-1}{\tau'} \end{bmatrix} \begin{bmatrix} x \\ y \\ \theta \\ v \\ \omega \end{bmatrix} + \begin{bmatrix} \frac{1}{\tau'} & 0 \\ 0 & \frac{1}{\tau'} \end{bmatrix} \begin{bmatrix} v_{ref} \\ \omega_{ref} \end{bmatrix}, \quad (7)$$

where v_{ref}, ω_{ref} are the reference velocity values that are given to the lower-level control. From these reference velocities, the individual wheel speeds can be computed by equation (3). This model allows us to consider the wheel acceleration dynamics instead of just the kinematic constraints.

III. CONTROL SYSTEM DESIGN

In this section, we will explore the design control architecture for the motion control of the robot. In this paper, we will divide the control system into two main blocks: a low-level and a high-level controller. The high-level block will

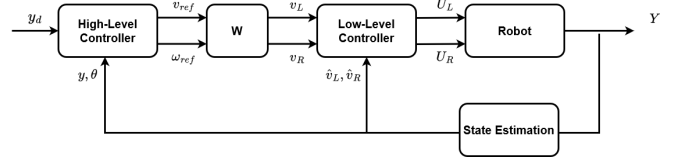


Fig. 2: Proposed Controller Architecture.

be responsible for the path following control by receiving the robot's pose and orientation and computing reference velocities handled by the low-level controller. This framework can be visualized in Figure 2.

A. Low-Level Controller

The low-level controller is responsible for controlling the wheel velocities so that the robot can achieve its desired speed.

1) *Proportional Integral Controller*: Consider the open-loop plant given by equation (5) and the let controller $H(s)$ be a PI controller described by equation:

$$H(s) = k_c \left(1 + \frac{1}{sT_i} \right) = k_c \frac{T_i s + 1}{sT_i}, \quad (8)$$

where k_c is the proportional gain and T_i is the integral time constant. The first design choice of the controller is to choose $T_i = \tau$, making the zero of the controller cancel the pole of the plant:

$$G(s)H(s) = k_c \frac{sT_i + 1}{sT_i} \frac{A}{\tau s + 1} = \frac{k_c A}{s\tau}. \quad (9)$$

Therefore, the closed-loop transfer can be derived as:

$$\frac{G(s)H(s)}{1 + G(s)H(s)} = \frac{1}{s \frac{\tau}{k_c A} + 1} \quad (10)$$

This results once again in a first-order system, this time with unitary gain, as the system tracks any input. The only remaining parameter to choose is the closed-loop time constant τ' , as this value will dictate how fast the control system can respond to changes in the reference signal. Then, the time constant of the closed-loop system is equal to:

$$\tau' = \frac{\tau}{k_c A}, \quad (11)$$

thus, the controller can be adequately tuned. Additionally, we can add a Feedforward signal $F(s)$ to the controller by inverting the plant:

$$F(s) = \frac{1}{G(s)} = \frac{1}{A} + \frac{s\tau}{A} = k_f + k_d s \quad (12)$$

and we will consider k_d to be equal to zero to keep the transfer function proper, and k_f will be $\lambda \frac{1}{A}$ where $\lambda \in [0, 1]$.

2) *Model Reference Adaptive Controller*: : Consider now that the motor model has parametric uncertainty and is described by the differential equation:

$$\dot{x} = ax + b(u - f(x)) \quad (13)$$

where a, b are unknown parameters and $f(x)$ is an uncertain nonlinear function

$$f(x) = \theta^T \phi(x), \quad (14)$$

with $\theta = [\theta_1, \theta_2]^T$ being a vector of unknown parameters and $\phi(x) = [1, \omega_w^2]^T$ a vector of known basis function. The goal of the Model Reference Adaptive Controller (MRAC) is to make the system described by equation (13) track a reference model given by:

$$\dot{x}_m = a_m x_m + b_m r \quad (15)$$

such that the tracking error $e(t) = x(t) - x_m(t) = 0$ as $t \rightarrow \infty$. To this end, let the control law be:

$$u = \hat{k}_x x + \hat{k}_r r + \hat{\theta}^T \phi(x), \quad (16)$$

and the adaptive law be:

$$\begin{aligned} \dot{\hat{k}}_x &= -e \gamma_x \text{sgn}(b) \\ \dot{\hat{k}}_r &= -e \gamma_r \text{sgn}(b) \\ \dot{\hat{\theta}} &= -e \Gamma_\theta \text{sgn}(b) \end{aligned} \quad (17)$$

that will update the parameters $\hat{k}_x, \hat{k}_r, \hat{\theta}$ such that the closed-loop dynamics:

$$\dot{x} = (a + b\hat{k}_x)x + b(\hat{k}_r r + (\hat{\theta} - \theta)^T \phi(x)) \quad (18)$$

match the reference model of equation (15).

IV. HIGH-LEVEL CONTROLLER

This section explores and describes various possibilities of high-level controllers for the path-following problem.

A. Linear Controller

Consider the dynamic model presented by the forward kinematics given by equation (2). We are interested in following only straight lines, which, without loss of generality, can be described as the line $y = 0$ in an arbitrary reference frame. Thus, since the problem is path-following and not trajectory tracking, the state x can be ignored as we are only concerned with following the geometric path. Furthermore, due to $y = 0$, the orientation of the line is also $\theta = 0$, and as such, our model can be linearized via small angle approximation:

$$\begin{bmatrix} \dot{y} \\ \dot{\theta} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & v(t) \\ 0 & 0 \end{bmatrix}}_A \begin{bmatrix} y \\ \theta \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ 1 \end{bmatrix}}_B \omega \quad (19)$$

where we consider that the velocity v is not a control variable and instead is a time-varying parameter of the model. Then, we can derive a full-state feedback controller:

$$\begin{bmatrix} \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & v(t) \\ 0 & 0 \end{bmatrix} \begin{bmatrix} y \\ \theta \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \end{bmatrix} [k_y \quad k_\theta] \begin{bmatrix} y \\ \theta \end{bmatrix} \quad (20)$$

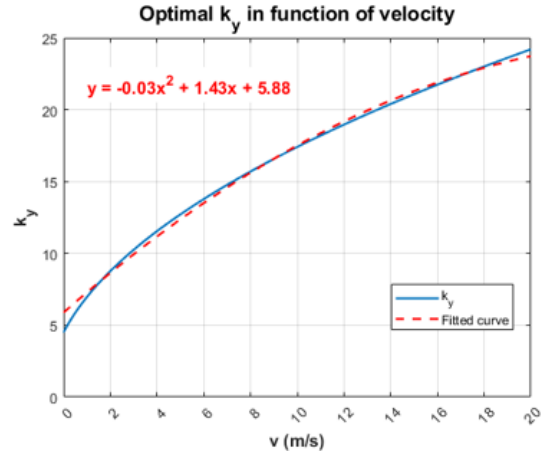


Fig. 3: Linear Regression of the values k_y

which results in the closed-loop eigenvalues being computed as:

$$\begin{aligned} \begin{vmatrix} 0 & v(t) \\ -k_y & -k_\theta \end{vmatrix} &= s^2 + sk_\theta + k_y v(t) = 0 \leftrightarrow \\ s &= \frac{-k_\theta \pm \sqrt{k_\theta^2 - 4k_y v(t)}}{2} \end{aligned} \quad (21)$$

and we will choose to have the imaginary part to be equal to zero, resulting in the expression for k_y being:

$$k_y = \frac{k_\theta^2}{4v(t)} \quad (22)$$

where the value of the real part is equal to $-\frac{k_\theta}{2}$, and as such we choose where to place the pole.

B. Linear Quadratic Regulator

Another option to formulate the linear control is through the optimal control lens. Let the system be described by the state-space model of equation (19) and let $Q, R \in \mathbb{R}^2$ are positive-definitive cost matrices. Then we can define the infinite-horizon Linear Quadratic Regulator (LQR) as:

$$\min J = \int_0^{+\infty} (x^T Q x + u^T R u) dt \quad (23)$$

with the closed-form solution being:

$$J = x^T P x \quad (24)$$

where $P \in \mathbb{R}^2$ can be obtained by solving the continuous algebraic Ricatti equation:

$$A^T P + P A - P B R^{-1} B^T P + Q = 0, \quad (25)$$

and the feedback gains are obtained by:

$$[k_y, k_\theta] = R^{-1} B^T P. \quad (26)$$

Note that the system is Linear Parameter Varying (LPV) model, and as such we would need to solve the Ricatti equation for every possible velocity. To deal with this issue, we compute the gains offline for multiple values of velocity

between $v \in [0, v_{max}]$ and perform a linear regression for obtaining an expression for the gains. The results were that the k_θ is constant and independent of the velocity whereas k_y follows a quadratic curve as seen in Figure 3.

C. Sliding Model Control

Another option for the high-level control is the Sliding Mode Control (SMC) methodology. This subsection we will describe a formulation for a SMC controller. Let the path y_d be defined by a function $f(x) : \mathbb{R} \rightarrow \mathbb{R}$. And the derivative at point x' is defined as $m = f'(x')$. Then, the reference orientation for the robot can be defined as:

$$\bar{\theta}_{ref} = \arctan(m). \quad (27)$$

Furthermore, the distance d from the current robot position (x', y') to the path $(x', f(x'))$ is computed as:

$$d = \frac{f(x') - y'}{\sqrt{1 + m^2}} \quad (28)$$

Then, assuming the robot is traveling at a velocity v we can generate a θ_{ref} that moves the robot towards the path:

$$\theta_{ref} = \bar{\theta}_{ref} + \gamma \arctan(kd) \quad (29)$$

where γ is the gain parameter and k is the parameter that dictates how steep the arctan function is.

This simple model enables the robot to follow the path by only making the robot orientation θ converge to θ_{ref} . In order to make this possible we design the sliding surface as:

$$s_\theta = \theta - \theta_{ref}, \quad (30)$$

and formulate the control law as:

$$\omega = -\beta \text{sgn}(s_\theta) \quad (31)$$

where β is the actuation effort. Let the Lyapunov function be defined as:

$$V = \frac{1}{2} s_\theta^2 \quad (32)$$

then, the existence of sliding mode is guaranteed as long as $\dot{V} = s_\theta \dot{s}_\theta < 0$. To prove the existence, let the first derivative of the surface be:

$$\begin{aligned} \dot{s}_\theta &= \dot{\theta} - \dot{\theta}_{ref} \\ &= -\beta \text{sgn}(s_\theta) - \dot{\theta}_{ref} \end{aligned} \quad (33)$$

which satisfies the sliding mode conditions as long as $\beta < |\dot{\theta}_{ref}|$.

D. Nonlinear Controller

The final option considered from the traditional approaches is a Lyapunov-based nonlinear controller.

To this end, consider the robot described by the dynamical model in equation (2), let $p(t) = [x(t), y(t)]^T \in \mathbb{R}^2$ be the pose of the robot, $R(t)$ be the rotation matrix from the body frame to the global frame, and $[v(t), \Omega(\omega(t))] \in SE(2)$ is the twist that defines the velocity of the vehicle, linear and angular, where the matrix $\Omega(\omega(t))$ is the skew-symmetric matrix associated to the angular velocity $\omega(t)$, defined as

$\Omega(\omega) = \begin{bmatrix} 0 & -\omega \\ \omega & 0 \end{bmatrix}$. And in this case, the control input is defined as:

$$u(t) = [v(t) \quad \omega(t)]^T \quad (34)$$

then we can rewrite the system dynamics as:

$$\begin{aligned} \dot{p} &= Rv \\ \dot{R} &= R\Omega \end{aligned} \quad (35)$$

Suppose that we want to drive the front of robot defined as:

$$y(t) = p(t) + R(t)\epsilon \quad (36)$$

where $\epsilon = [\epsilon_1, 0]$ is the distance from the front of to the robot to the center of rotation of the body frame, and now we can define and define the body-fixed path following error:

$$e(t) = R^T(t)(y(t) - y_d(\gamma(t))). \quad (37)$$

As the goal of the path-following problem is to make the error $e(t) \rightarrow 0$ as $t \rightarrow \infty$, we propose the candidate Lyapunov Function $V = \frac{1}{2} e^T e$. Let the time-derivative of the error be defined as:

$$\dot{e} = -\Omega e + \Delta u - R^T \frac{\partial y_d}{\partial \gamma} \dot{\gamma} \quad (38)$$

can formulate the control law as follows:

$$u(t) = \Delta^{-1}(R^T \frac{\partial y_d}{\partial \gamma} v_d - K e)$$

where $\Delta = \begin{bmatrix} 0 & \epsilon_1 \\ 0 & 0 \end{bmatrix}$, $\dot{\gamma} = v_d$ and K is the feedback gain. It follows that the derivative of the Lyapunov function is:

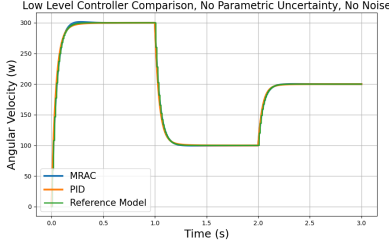
$$\begin{aligned} \dot{V} &= e^T \dot{e} \\ &= -e^T \Omega e - e^T K e \end{aligned} \quad (39)$$

where $e^T \Omega e = 0, \forall e$ because Ω is a skew-symmetric matrix and $e^T K e < 0$, thus $e = 0$ is an exponentially stable equilibrium point.

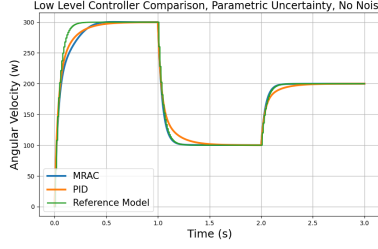
E. Proximal Policy Optimization

The final option for the high-level controller is to use Reinforcement Learning Methods, in this case Proximal Policy Optimization which is very versatile actor-critic method. To this end, we must formulate the path-following problem as a Markov Decision Process (MDP).

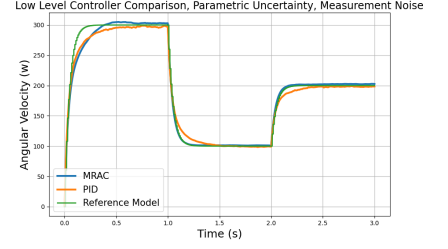
A MDP can be defined by a tuple $(\mathbb{S}, \mathbb{A}, \mathbf{P}, R, \gamma)$, where $\mathbb{S}$ is the set that contains all the possible states s , and $\mathbb{A}$ is the state that contains all the actions a on the action space; $\mathbf{P} : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{S}$ is the state transition probability function, $R : \mathbb{S} \times \mathbb{A} \times \mathbb{S} \rightarrow \mathbb{R}$ is the reward function, mapping the state transitions into a scalar reward; The discount factor $\gamma \in [0, 1]$ changes the weight given to future rewards.



(a) No Parametric Uncertainty and No Measurement Noise.



(b) Parametric Uncertainty and No Measurement Noise.



(c) Parametric Uncertainty and Measurement Noise.

Fig. 4: Software in The Loop Results for Low-Level Controller

1) *State Space*: For the case of path following we will consider the state space to be the tuple defined by the tuple:

$$\mathbb{S} = \underbrace{\mathbb{R}^{3 \times n}}_{\bar{x}} \times \underbrace{\mathbb{R}^{2 \times n}}_u \times \underbrace{\mathbb{R}^{2 \times n}}_{y_d}, \quad (40)$$

where $\bar{x} = [x, y, \theta]^T$ corresponds to the vector containing both the robot's pose and orientation, $u = [v, \omega]^T$ is the robot linear and angular velocities and $y_d = [y_{ref}, \theta_{ref}]$ corresponds to the path. In addition, we perform frame stacking by giving the n most recent values for each variable.

2) *Action Space*: As in most previous approaches, we will consider that the robot's linear velocity is not handled by the high-level controller directly and instead is a function of time $v(t)$. Then, the only control variable is the angular velocity $\omega(t)$ and, as such the action space is described as:

$$\mathbb{A} = [-1, 1] \quad (41)$$

and the angular velocity ω is a function of action a :

$$\omega_t = \lambda a_t \quad (42)$$

where λ is an hyperparameter that scales the value of the action to the desired angular velocity. This is particularly useful as policy gradient algorithms work specifically well when the action space is symmetric and centered in zero, as the policy generally approximates a probability distribution function with the same conditions.

3) *Reward*: The reward function is set so that the value function resembles the LQR cost function. This design choice is due to the fact that the LQR cost function describes the basic goal of the path-following problem; the quadratic cost function is a natural and easier function to approximate, and it is proven that reinforcement learning algorithms can learn

the value function in a finite number of samples. Thus, the reward function is described as:

$$R(s, a, s)' = -(e_t^T Q e_t + R \omega_t^2) \quad (43)$$

where $e_t = [y_d - y, \theta_d - \theta]$ is the path following error, $Q \in \mathbb{R}^2$ is a positive definite matrix, and R is a real positive number. Note that unlike LQR, where the problem is posed as a minimization problem, in the RL framework, the problem is posed as a maximization of the expected reward, so we change the sign of the reward.

V. SIMULATION RESULTS

To validate the proposed architecture, the system is implemented in software. This section presents the main simulation results, first comparing low-level and high-level controllers.

A. Motor Control

To test the efficacy of the low-level controllers, we perform software in the loop tests where the motors are subjected to a series of step references. The tests have three different scenarios, one where the simulated motor's model and the identified model used to tune the controllers are the same. The second scenario introduces parametric uncertainty of up to 25% both in the gain and time constant and in the final scenario, alongside the uncertainty, we also introduce measurement noise in the form of a normal distribution $\mathcal{N}(0.01\omega, 0.01\omega)$. The results can be visualized in Figure 4, and the Mean Square Tracking Error (MSE) for each scenario is presented in Table I. Note that the MRAC gains were initialized with the matching condition from the identified model.

From the results, it is possible to infer that if the simulation model matches the identified model, both controllers match almost perfectly the ideal reference model. In the case where

TABLE I: Mean Square Error (ω^2) from the Low-Level Controller Tests. The Columns marked with $\dagger$ are the results from $t > 1$, i.e., after the MRAC gains converge.

	MRAC	PID	MRAC †	PID †
No Noise, No Uncertainty	2.80	1.6	0.78	0.45
No Noise, Uncertainty	239.00	113.43	0.42	31.53
Noise and Uncertainty	230.14	149.13	3.45	30.86

TABLE II: Mean Absolute Error ($|y|$) from the Low-Level Controller Tests.

	Linear	SMC	NLC	PPO
No Noise, No Uncertainty	0.0113	0.0106	0.0025	0.0078
No Noise, Uncertainty	0.0118	0.0102	0.0027	0.0084
Noise and Uncertainty	0.0155	0.0146	0.00884	0.0224

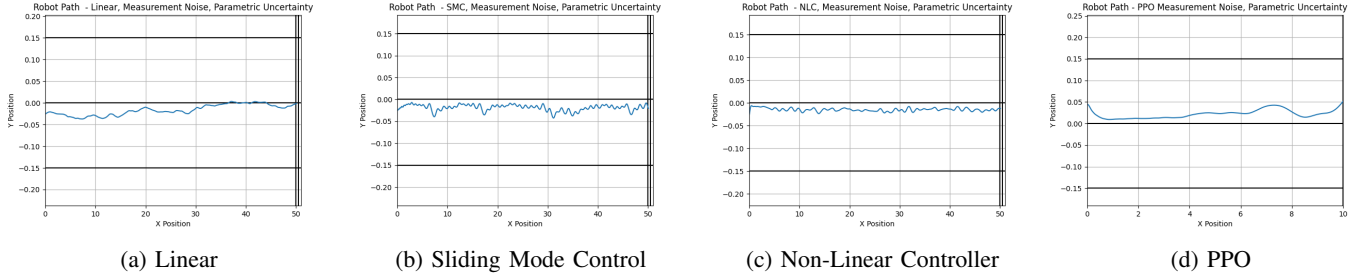


Fig. 5: Software in The Loop Results for Path Following Controller

parametric uncertainties are introduced, the performance significantly degrades as the closed-loop system is much slower than the reference model. However, after the first step reference, the MRAC parameters converge, and the controller matches the reference model again, unlike PID, whose poor performance is consistent in the subsequent changes in the reference signal. Finally, when measurement noise is included, the controller performance worsens, but both controllers are robust enough to track the reference adequately.

B. Path-Following Control

The final test was to test the path following the controller on a simulation scenario. In this scenario, the robot's kinematics and motor dynamics are simulated, and actuation limits and state estimation uncertainty are simulated. The path the robot must take is a straight line $y = 0$ and the initial condition is $x_0 = 0, y_0 = 0.04$. The robot's desired speed is a ramp signal $v_d(t) = 5t$ corresponding to an acceleration $a = 5\text{m/s}^2$, and the control loop runs at 100Hz. Some example paths can be visualized in Figure 5, and the Mean Absolute Error (MAE) for all performed tests can be seen in Table II.

From all the controllers, the one with the best performance in all cases is the Non-Linear Controller (NLC). However, it is possible to see that due to the measurement bias, there exists some steady-state error that the controller can not overcome in the scenario with measurement noise. This issue could be mitigated by adding an integral term to the controller. Both Linear and Sliding Mode Controllers are more robust to the introduction of noise as their performance does not degrade as much, nonetheless in terms of absolute error, both controllers are almost interchangeable, where the main difference is that the linear controller is the one that achieves the most smooth paths. The SMC could be improved by increasing the control loop frequency and by adding a filtering term with the derivative of θ_{ref} to the sliding surface. The linear controller is at almost its performance limit, however it could be improved by accounting for the poles of the motors in the LQR calculations (instead of using just the kinematics). Finally, PPO achieved some mixed results, while it performed better than all controllers bar NLC without noise, when adding this element it was the controller that degraded the most. The improvements should be on accounting for these noise sources in the training process and improvements on the reward function/state space.

Finally, an interesting result is that for all controllers the performance with and without parametric uncertainties are similar for controllers. This is a very good result as it shows that the MRAC scheme is working as intended and can successfully overcome the modeling uncertainty.

VI. CONCLUSION

In this paper we explore the development of a path-following framework for high-speed differential robots. First, we explore the kinematics and dynamics for the wheeled robot and give two different kinodynamics models. Then, we divide the path-following framework into two main components: a low-level controller for the control of the motor velocities and then a high-level controller responsible for the path-following. In the low-level controller, we explore the possibility of using PID and Model Reference Adaptive control, and the results validate by options, and it was demonstrated that the MRAC scheme can successfully account for parametric uncertainties in the motor's model. Finally, we explore four different approaches for the path-following controller: Linear Quadratic Regulator, Sliding Model Control, Non-Linear Controller, and Proximal Policy Optimization. While all controllers were shown to accomplish the task, the non-linear controller performed better. Several improvements can still be made to the framework: addressing the state estimation bias by adding a Kalman Filter and fusing the wheel odometry data with an IMU, improving the existent controllers and exploring more techniques related to the RL component. Furthermore, it is paramount to perform more hardware tests, specifically system identification with Gaussian-Process Regression.